Automated Software Engineering (2025) 32:45
https://doi.org/10.1007/s10515-025-00525-5

Improving prompt tuning-based software vulnerability assessment by fusing source code and vulnerability description

Jiyu Wang[1] · Xiang Chen[1,2] · Wenlong Pei[1] · Shaoyu Yang[1]

Received: 30 June 2024 / Accepted: 23 April 2025

Abstract
To effectively allocate resources for vulnerability remediation, it is crucial to prioritize vulnerability fixes based on vulnerability severity. With the increasingnumber of vulnerabilities in recent years, there is an urgent need for automated methods for software vulnerability assessment (SVA). Most of the previous SVA studies mainly rely on traditional machine learning methods. Recently, fine-tuning pre-trained language models has emerged as an intuitive method for improving performance. However, there is a gap between pre-training and fine-tuning, and their performance heavily depends on the dataset's quality of the downstream task. Therefore, we propose a prompt tuning-based method PT-SVA. Different from the fine-tuning paradigm, the prompt-tuning paradigm involves adding prompts to make the training process similar to pre-training, thereby better adapting to downstream tasks. Moreover, previous research aimed to automatically predict severity by only analyzing either the vulnerability descriptions or the source code of the vulnerability. Therefore, we further consider both types of vulnerability information for designing hybrid prompts (i.e., a combination of hard and soft prompts). To evaluate PT-SVA, we construct the SVA dataset based on the CVSS V3 standard, while previous SVA studies only consider the CVSS V2 standard. Experimental results show that PT-SVA outperforms ten state-of-the-art SVA baselines, such as by 13.7% to 42.1% in terms of MCC. Finally, our ablation experiments confirm the effectiveness of PT-SVA's design, specifically in replacing fine-tuning with prompt tuning, incorporating both types of vulnerability information, and adopting hybrid prompts. Our promising results indicate that prompt tuning-based SVA is a promising direction and needs more follow-up studies.

Keywords Software vulnerability assessment · Prompt tuning · Bi-modal information · Hybrid prompt · Pre-trained language model

1 Introduction

Software vulnerabilities pose serious security and economic risks, necessitating effective automatic detection methods (Wang et al. 2024; Ren et al. 2024; Liu et al. 2024a, b;

Extended author information available on the last page of the article

Published online: 03 May 2025

Cai et al. 2024; Lu et al. 2024; Zheng et al. 2020; Chen et al. 2019; Garg et al. 2022). As the number of reported vulnerabilities increases, prioritizing their repair becomes critical due to limited resources. For instance, the widely-publicized Apache Log4j2 vulnerability (CVE-2021-44228) (Apache n.d.) underscored the urgent need to address high-risk vulnerabilities swiftly to prevent severe system compromise. The Common Vulnerability Scoring System (CVSS) (CVSS n.d.) is a standard tool for assessing vulnerability severity, but its reliance on expert knowledge often delays scoring, hindering timely remediation (Feutrill et al. 2018). This calls for automated tools to help developers prioritize vulnerability fixes efficiently.

Researchers have developed Software Vulnerability Assessment (SVA) models to predict severity based on different types of vulnerability information, such as vulnerability descriptions (Han et al. 2017; Kudjo et al. 2020; Liu et al. 2019) or vulnerability code characteristics (Hao et al. 2023; Le et al. 2021). However, integrating both the source code and vulnerability description (bi-modal information) may improve prediction accuracy. Based on the analysis of previous SVA studies, most methods rely on traditional machine learning methods. One promising way to enhance SVA performance is by using pre-trained language models (PLMs) with the fine-tuning paradigm (Li et al. 2023; Han et al. 2021; Yang et al. 2022; Liu et al. 2023; Yang et al. 2023). PLMs undergo extensive pre-training on large corpora and are then fine-tuned for specific downstream tasks, achieving notable success in many applications (Liu et al. 2022; Howard and Ruder 2018; Liu et al. 2016). However, there are significant differences between pre-training tasks and vulnerability assessment tasks, which are modeled as a classification problem (Wang et al. 2022). Firstly, learning domain-specific knowledge is challenging (Ren et al. 2024). Secondly, fine-tuning PLMs requires a large amount of high-quality data to perform well (Zhang et al. 2021; Lester et al. 2021; Gu et al. 2022). These challenges highlight the need for tailored methods to effectively leverage PLMs for SVA. As shown in Fig. 1(a), the pre-training process involves training the model to predict masked tokens. However, when applying PLMs to the SVA task, issues arise. Our chosen PLM CodeT5 (Wang et al. 2021) is a sequence-to-sequence model. Fine-tuning treats the bi-modal information in a static manner, relying on predefined separators (as illustrated in Fig. 1(b)), which limits the model's ability to dynamically adapt to the differences in how code and text should

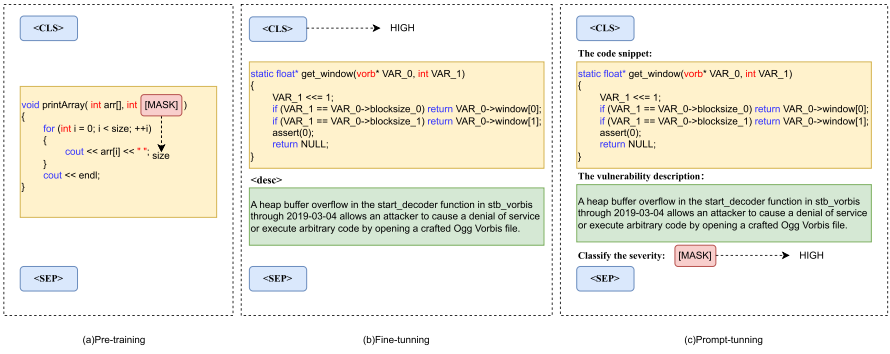

Fig. 1 Illustration on the process of pre-training, fine-tuning, and prompt tuning for the SVA task

be processed. In contrast, prompt tuning incorporates prompts that actively guide the model to process bi-modal information more effectively, better aligning the task with the model's pre-training objectives.

The recent emergence of the prompt tuning paradigm can alleviate some of the issues faced by the fine-tuning paradigm (Liu et al. 2023; Yu et al. 2023). Applying prompts to PLMs enables the model to simulate the pre-training process, thus better handling downstream tasks. This is achieved by adding natural language and masked tokens to create new inputs, mimicking the pre-training tasks and effectively extracting knowledge for the downstream task (Guo et al. 2019; Li and Liang 2021; Shen et al. 2021). As shown in Fig. 1(c), to predict the severity of software vulnerabilities, we add natural language prompts, such as "The vulnerability description:" to distinguish the vulnerability description, and "The code snippet:" to distinguish the source code. For prediction prompts, we add "Classify the severity: [MASK]," where [MASK] is the masked token used to output the predicted word. Our designed verbalizer then maps the predicted word to the severity category, resembling the PLM pre-training process and fully extracting knowledge related to SVA.

Based on the above motivation analysis, we propose the prompt tuning-based method PT-SVA. Most of the SVA datasets used in previous research have CVSS scores based on the CVSS v2 standard. However, we observed that, as of July 13, 2022, the NVD no longer generates new information for CVSS v2. Existing CVSS v2 information will remain in the database, but the NVD will no longer actively populate CVSS v2 for new CVEs (the differences between CVSS v2 and CVSS v3 can be found in Section 2.1). Therefore, in our study, we crawled data based on the CVSS v3 standard. By following common practice (Le et al. 2021), we also removed blank lines, comments, and leading whitespace to simplify the vulnerability source code, ultimately creating a high-quality SVA dataset. In the model-building phase, we use bi-modal information as input and construct prompt templates as new inputs. Then we apply prompt tuning to the PLM CodeT5 (Wang et al. 2021) to achieve the downstream task SVA. In the model prediction phase, we input source code and vulnerability descriptions of the target vulnerability into the trained hybrid prompt template and the fine-tuned model. Then the verbalizer maps the predicted words to the corresponding severity categories.

To validate the effectiveness of our proposed method PT-SVA, we conducted comparative experiments with five state-of-the-art SVA baselines (Le et al. 2019; Le and Babar 2022) (i.e., CWM_{NB}, CWM_{SVM}, CWM_{XGB}, Fun_{RF} and Fun_{LGBM}) and fine tuning-based baselines. We used common evaluation metrics (i.e., Accuracy, Precision, Recall, F1 score, and MCC) for a comprehensive performance evaluation. The experimental results show that PT-SVA outperforms these baselines. For example, in terms of the MCC metric, we achieve a performance improvement of 13.7% to 42.1%. Moreover, our ablation experiments further demonstrate the effectiveness of using bi-modal information, and hybrid templates for PT-SVA.

The promising results of our study provided two feasible directions for future SVA studies, which calls for more follow-up studies. The first direction is to consider more effective prompt tuning-based methods for the SVA task, and the second direction is to consider more information on software vulnerabilities (such as the structural information of source code).

To the best of our knowledge, the contributions of our study can be summarized as follows:

- **Dataset.** We constructed a high-quality SVA dataset with CVSS v3 standard, which can facilitate future SVA studies.
- **Method.** We are the first to apply prompt tuning to bi-modal inputs (both source code and vulnerability descriptions) for vulnerability severity prediction and propose the method PT-SVA. PT-SVA leverages the pre-trained language model CodeT5, incorporates bi-modal information input, and employs our designed hybrid prompts for improving the SVA performance.
- **Evaluation.** Evaluation results on our constructed SVA dataset show the effectiveness of PT-SVA when compared with state-of-the-art SVA baselines and our customization designs for PT-SVA via a set of ablation studies.

Open Science To support the open science community, we share the experimental subjects, code scripts, and detailed results of our research in a GitHub repository (https://github.com/1-001/PT-SVA).

Paper Organization Section 2 introduces the background of our study, including software vulnerability assessment, Prompt Tuning and pre-trained Language Model CodeT5. Section 3 presents the framework of PT-SVA and details of each stage. Section 4 describes our experimental setup, including the research questions and their design motivations, our constructed dataset, performance metrics, baselines, and experimental settings. Section 5 discusses our experimental results. Section 6 analyzes potential threats to the validity of our experimental results. Section 7 summarizes the related work and highlights the novelty of our study. Section 8 concludes our study and discusses potential future work.

2 Background

2.1 Software vulnerability assessment

Common Vulnerabilities and Exposures (CVE) is a publicly available database that provides standardized names for known vulnerabilities. CVE details include information (such as the vulnerability ID, standardized description, and CVSS score), which help developers understand and prioritize vulnerabilities effectively. Prioritizing vulnerabilities is crucial because high-risk vulnerabilities need to be addressed as soon as possible. The longer the delay in fixing a vulnerability, the greater the potential threat. Moreover, the number of vulnerabilities has been increasing in recent years, while security experts and remediation resources remain relatively limited. Therefore, it is not feasible to fix vulnerabilities in sequential order. Consequently, SVA is a critical step in remediation, helping security experts prioritize high-risk vulnerabilities. The Common Vulnerability Scoring System (CVSS) is the most commonly used framework for SVA. Maintained by security experts, this framework considers multiple aspects of a vulnerability, assigns weights to each aspect, and generates a severity score ranging from 0 to 10, with higher scores indicating more severe vulnerabilities.

The CVSS score comprises three main parts: base metrics, temporal metrics, and environmental metrics. However, obtaining temporal and environmental metrics from code or descriptions is relatively difficult. Base metrics are the most important part, and previous studies have primarily calculated CVSS scores using these metrics. CVSS v2 had three severity categories: Low (0.1∼3.9), Medium (4.0∼6.9), and High (7.0∼10.0). CVSS v3, released in 2016, added a critical category on top of CVSS v2, resulting in four categories: Low (0.1∼3.9), Medium (4.0∼6.9), High (7.0∼8.9), and Critical (9.0∼10.0). Additionally, CVSS v3 optimized the evaluation elements from CVSS v2. Recent vulnerability reports predominantly use CVSS v3 scores, making CVSS v2 scores largely obsolete. Therefore, the CVSS scores in our dataset adhere entirely to the CVSS v3 standard. In our SVA task, we aim to directly predict the severity category, providing security experts with valuable reference points for remediation efforts.

2.2 Prompt tuning

Prompt tuning is a method that allows downstream tasks to mimic pre-training tasks by designing prompt templates to improve adaptability. This is mainly achieved by adding natural language to the input, reconstructing the input information, and then letting the PLM predict the content in the masked token, as shown in Fig. 1(c). In the SVA task, by adding natural language information and constructing prompt templates, the PLM can better extract vulnerability information and distinguish the source code and the vulnerability description of the vulnerability. Specifically, this involves using the template $f_{\text{prompt}}(x)$ to transform the original input into a new input x'. In the template, there are two slots: one for the input data and one for the answer, used for data input and answer prediction, respectively. Then, through the verbalizer, the predicted word is mapped to the target category. For the verbalizer, there are one-to-one and one-to-many types, used to map the label word with the highest probability to the target category. When setting up the verbalizer's label words, semantically similar words should be grouped under the same category. The principle of the verbalizer V mapping word W to Y is:

$$V : W \rightarrow Y$$

Prompt templates can be hard prompts or soft prompts. A hard prompt involves manually crafting natural language phrases or templates to guide the pre-trained language model (PLM) in performing specific tasks. These prompts are static and do not change during the training process. A soft prompt, on the other hand, involves adding learnable embeddings to the input tokens. Unlike hard prompts, soft prompts are not fixed and can be optimized during the training process. These prompts are represented as continuous vectors, allowing the model to adapt and fine-tune them to better suit the specific downstream task. For the SVA task, natural language can be added to introduce the source code and vulnerability description, as shown in Fig. 1(c). It is through the prompt templates and verbalizers that pre-trained language models are better suited to downstream tasks.

Recent research by Wang et al. (2022) applied prompt tuning to code intelligence tasks, including defect prediction, code summarization, and code translation. Their

empirical results showed better performance with prompt tuning. Recently, Yang et al. (2024a) applied prompt tuning to the Stack Overflow title generation task and still achieved a promising performance. These studies have demonstrated the effectiveness of the prompt-tuning paradigm when compared with the fine-tuning paradigm. In our research, we aim to apply prompt tuning to SVA and investigate the effectiveness of our proposed prompt tuning-based method PT-SVA.

2.3 CodeT5

The studies of Ruan et al. (2023) and Niu et al. (2023) demonstrate the effectiveness of using the pre-trained language model CodeT5 (Wang et al. 2021) for downstream tasks. CodeT5 is a pre-trained encoder-decoder model based on the T5 (Raffel et al. 2020) architecture, but CodeT5 performs some programming language-specific techniques when fine-tuning, such as marking and masking code snippets so that the model can better understand them. The structure and syntax of the code make CodeT5 more suitable for handling program code-related tasks. CodeT5 uses a code-specific BPE (Byte Pair Encoding) tokenizer to better handle special symbols and subwords in code.

The encoder consists of multiple "chunks", each of which consists of two subcomponents: a self-attention layer (Cheng et al. 2016) and a small feed-forward neural network. Self-attention is calculated based on a set of queries (Q), keys (K), and values (V). The dot product between the query and the key is calculated and then divided by $\sqrt{d_k}$, and the softmax function is applied to get the weight of the corresponding value. All attention mechanisms in Transformer are split into independent "heads", whose outputs are concatenated before further processing. Each layer of the encoder contains a fully connected feedforward network (FFN). The calculation formula for self-attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

The structure of the decoder is similar to that of the encoder, but its self-attention mechanism is modified and uses an autoregressive form where the model only focuses on past outputs. Additionally, layer normalization is applied to the input of each subcomponent, and residual connections add the input of each subcomponent to its output, which facilitates deep architectures. Note that CodeT5 uses a bi-modal input encoder to encode programming language and natural language information into a unified representation. This encoder consists of two sub-encoders, one for encoding programming language information and the other for encoding natural language information. Wang et al. (2021) applied CodeT5 to comprehension tasks (defect detection and clone detection) and found that it outperformed all baselines in defect detection tasks. In clone detection tasks, the CodeT5 model achieved results comparable to baseline models, demonstrating that the encoder-decoder framework of CodeT5 can still adapt well to comprehension tasks. During the prompt tuning phase for downstream tasks, convert it into cloze-style tasks (Nie et al. 2023).

3 Our proposed method PT-SVA

The overall framework of PT-SVA is shown in Fig. 2, which is divided into three main phases: the preprocessing phase, the model construction phase, and the model prediction phase. In the remainder of this section, we present the details of these three phases.

3.1 Preprocessing Phase

In this phase, we perform data preprocessing to obtain a high-quality SVA dataset. In our study, we chose the dataset MegaVul shared by Ni et al. (2024) as our experimental subject and only selected vulnerable functions for the C/C++ programming language. After analyzing the MegVul dataset, We found that the vulnerability score vectors were not all in the CVSS v3 standard. Therefore, we crawled the corresponding data under the CVSS v3 standard and finally constructed an SVA dataset with the CVSS v3 standard. Moreover, after analyzing the vulnerability code, we found that some elements (such as blank lines, comments, and leading whitespace) could potentially affect model prediction performance. These elements were removed due to the input length limitations of pre-trained language models, as we want the model to learn as much relevant information as possible. Simplifying the source code does not affect its functionality. Additionally, the model might mistakenly recognize comments as code, which could lead to incorrect predictions. These preprocessing steps help make the source code more suitable as input for the SVA task.

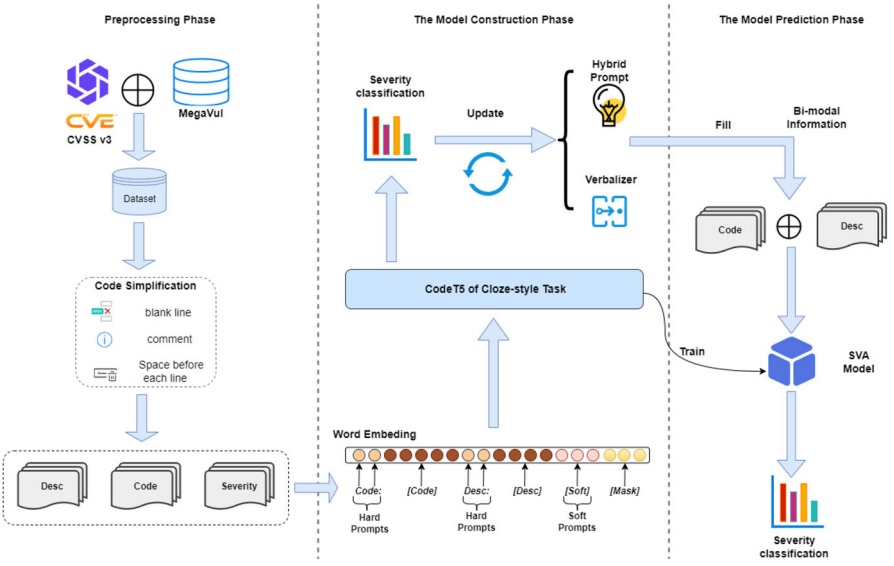

Fig. 2 Overview of our proposed method PT-SVA

3.2 Model construction phase

In our proposed method PT-SVA, we aim to train the software vulnerability assessment model by considering both types of vulnerability information (i.e. source code and vulnerability descriptions) and then apply prompt tuning to PLM to achieve the downstream task of SVA.

We distinguish these two types of vulnerability information through hard prompts by adding the phrases: "The code snippet:" and "The vulnerability description:" For vulnerability severity prediction, we create soft prompts by using the [SOFT] tag. Based on hybrid prompts (i.e., a mixture of hard and soft prompts), our SVA model is fine-tuned using the pre-trained Transformer model CodeT5 (Wang et al. 2021), which has been widely used in previous software engineering studies (Ciniselli et al. 2024; Yang et al. 2024a, b; Lu et al. 2023). Specifically, we combine the processed data and prompt templates to transform them into new inputs, which are subsequently fed into a PLM. The model leverages its pre-trained knowledge to predict mask locations in the input, and then the prediction results are mapped to actual labels, converting the downstream task into a masked prediction task.

3.2.1 Prompt template and verbalizer construction

Designing high-quality prompt templates for downstream tasks of pre-trained language models is challenging and often requires multiple attempts to obtain good templates. There are three different types of prompt templates: hard prompts, soft prompts, and hybrid prompts. The verbalizer converts the predictions at the [MASK] position into classification labels, with the vocabulary words needing to be closely related to the categories. Verbalizers come in two types: one-to-one and one-to-many.

Hard prompt The hard prompt (Liu et al. 2023) is the predefined, handcrafted text that serves as an interpretable word or token, often designed by experienced domain experts. This method is intuitive and natural. The template primarily has two types of slots: input slots and answer slots. Since we consider both types of vulnerability information in our SVA task, we use two input slots. We can define the hard prompt for the SVA task as follows:

$$f_{hard} = \text{The code snippet: [X] The vulnerability description: [Y] Classify the severity: [Z]} \tag{2}$$

The input slot [X] will be filled with the source code, and the input slot [Y] will be filled with the vulnerability description. The PLM predicts the probability distribution of label words at position [Z], with the label word having the highest probability being the intermediate generated answer by the PLM.

Soft prompt The soft prompt (Liu et al. 2023; Tsimpoukelli et al. 2021; Han et al. 2022) is more flexible, with the generation of prompts being learned as a task itself. This essentially transforms the generation of prompts from a human-driven, trial-and-error process (discrete) into a machine-driven learning and experimentation process

(continuous). Soft prompts lack interpretability, cannot be viewed or edited in text form, and are continuous embeddings optimized during training. However, they overcome the difficulty of creating an appropriate hard prompt, allowing the model to perform more freely. For our investigated SVA task, we define the soft prompt as follows:

$$f_{soft} = [SOFT][X][SOFT][Y][SOFT][Z] \quad (3)$$

In our research, [SOFT] tokens are initialized using embeddings of natural words through a hard prompt template (Han et al. 2022). For example, the first [SOFT] token can be initialized as "The code snippet:", the second [SOFT] token can be initialized as "The vulnerability description:", and the third [SOFT] token can be initialized as "Classify the severity:". This aims to ensure the replication of our experiments and can also help in finding the optimal embeddings (Zhong et al. 2021).

Hybrid prompt The hybrid prompt (Li et al. 2023) combines both hard prompt and soft prompt, leveraging the advantages of both. Tokens in a hard prompt are not optimized during training, so they generally represent important tokens relevant to the downstream task. In contrast, soft prompt tokens can be optimized throughout the training process and usually represent less critical tokens. In our investigated SVA task, we consider "The code snippet:" and "The vulnerability description:" as important tokens because they are task-relevant and help the pre-trained language model distinguish two types of vulnerability information. Therefore, we retain these as hard prompt tokens. On the other hand, for the token "classify the severity:", we use a soft prompt represented by [SOFT]. This is because this phrase has many alternative tokens, such as "differentiate the severity:" or "the severity of this vulnerability is:". Thus, we use a soft prompt to allow the model to generate the prompt itself. An example of a hybrid prompt is shown in Fig. 1(c). We define the hybrid prompt for the SVA task as follows:

$$f_{hybrid} = \text{The code snippet: [X] The vulnerability description: [Y] [SOFT] [Z]} \quad (4)$$

Verbalizer The Verbalizer (Schick and Schütze 2021) transforms predictions at the [MASK] position for words in the vocabulary into classification labels. These labels constrain the output space of the PLM, meaning the model's output probabilities are focused solely on these label words. Each target class can have one or more label words. By mapping the label word with the highest probability to the target class label through the verbalizer, we obtain the final prediction. In the SVA task, target classes can be directly used as label words or similar label words can be added.

The vocabulary size of the pre-trained language model is large, and with the verbalizer, we can focus only on these specific label words, making the model's output consistent with the target task and enhancing interpretability. Based on the number of labels corresponding to the target categories, verbalizers can be divided into one-to-one and one-to-many types. We aim to explore the impact of the number of labels on model performance. For the SVA task, we define one-to-one and one-to-many as

follows:

$$Verbalizer_{O2O} = \begin{cases} \text{“}LOW\text{”}: & [\text{“}low\text{”}] \\ \text{“}MEDIUM\text{”}: & [\text{“}medium\text{”}] \\ \text{“}HIGH\text{”}: & [\text{“}high\text{”}] \\ \text{“}CRITICAL\text{”}: & [\text{“}critical\text{”}] \end{cases} \tag{5}$$

$$Verbalizer_{O2M} = \begin{cases} \text{“}LOW\text{”}: & [\text{“}low\text{”}, \text{“}slight\text{”}] \\ \text{“}MEDIUM\text{”}: & [\text{“}medium\text{”}, \text{“}moderate\text{”}] \\ \text{“}HIGH\text{”}: & [\text{“}high\text{”}, \text{“}severe\text{”}] \\ \text{“}CRITICAL\text{”}: & [\text{“}critical\text{”}, \text{“}significant\text{”}] \end{cases} \tag{6}$$

Regarding the implementation of the one-to-one and one-to-many mappings, our approach is based on vocabulary projection, which uses the LM_head in CodeT5 rather than training a new multi-layer perceptron (MLP). Specifically, in the one-to-one setting, each class (label) is mapped to a single label word, where we directly extract the corresponding token from CodeT5's vocabulary and use its logits to compute the class probability. In the one-to-many setting, each class is associated with multiple label words, and we first obtain the logits of these words before applying mean pooling to aggregate their scores into a final class prediction. This design ensures that the mappings are handled efficiently while preserving the integrity of the PLM's inherent knowledge. This approach allows us to leverage the PLM's existing vocabulary and contextual understanding without the need for external transformation layers such as MLP.

3.2.2 Prompt tuning on CodeT5

During model training, since the SVA task is modeled as a classification task, to better observe the loss in the validation set and facilitate the model optimization, we use the cross-entropy loss function. The cross-entropy formula is defined as follows:

$$\mathcal{L} = \frac{1}{N}\sum_{i} = -\frac{1}{N}\sum_{i}\sum_{c=1}^{M} y_i log(p_i) \tag{7}$$

where M represents the number of sample classes, y_i denotes the ground-truth labels, and p_i denotes the predicted labels. The cross-entropy loss function measures the difference between the ground-truth labels and the predicted labels. When the predicted label matches the ground-truth label, the value of the cross-entropy loss function is 0. The larger the value, the greater the difference between the predicted result and the ground-truth result.

3.3 Model prediction phase

During the model prediction phase, we extract source and vulnerability descriptions from the target vulnerability. We combine this data with the trained hybrid prompt template to form new inputs, which are then fed into our trained SVA model to classify the severity of the target vulnerability.

4 Experimental setup

4.1 Research questions

To evaluate PT-SVA, we aim to answer the following three research questions:

RQ1: How effective is PT-SVA in SVA?

Motivation In RQ1, we want to demonstrate the competitiveness of our proposed method PT-SVA by considering state-of-the-art SVA baselines and the controlled methods based on the fine-tuning paradigm. Moreover, we consider five automatic evaluation metrics (i.e., Accuracy, Precision, Recall, F1 score, and MCC) for a comprehensive evaluation.

RQ2: Can using both types of vulnerability information improve the performance of PT-SVA in SVA?

Motivation Our proposed method PT-SVA utilizes both source code and vulnerability descriptions as input, combined with our designed prompt templates to form new inputs. These are then fed into the model as bi-modal information. Therefore, in RQ2, we aim to investigate whether this bi-modal input setup achieves the best performance for PT-SVA. Moreover, We want to compare different modality settings (inputting only source code or only vulnerability descriptions) to determine which input mode contributes most to the performance of PT-SVA.

RQ3: How do different prompt settings affect the performance of prompt tuning for PT-SVA?

Motivation Selecting appropriate prompt templates and verbalizers for downstream tasks is a challenging and open problem. In RQ3, we want to conduct ablation experiments to demonstrate the competitiveness of using hybrid prompts in our proposed method PT-SVA. Moreover, we want to examine the impact of the number of label words in the verbalizer on the performance of PT-SVA.

4.2 Dataset

Our study utilizes MegaVul (Ni et al. 2024) as the initial dataset. MegaVul comprises a total of 17,380 vulnerabilities collected from 992 open-source repositories, covering 169 different vulnerability types disclosed between January 2006 and October 2023. In previous SVA studies, the most widely used dataset is Big-Vul (Fan et al. 2020). Compared to the BigVul dataset, MegaVul contains 64.8% more vulnerabilities and includes a wider variety of vulnerability types. Additionally, BigVul only covers data from 2003 to 2019 and is of lower quality, with incomplete functions, incorrectly merged functions, and missing commit messages. Unlike BigVul, which uses regular expressions for function extraction, MegaVul employs a complex, syntax rule-based parser tree to separate functions. The specific differences between these two datasets are shown in Table 1, demonstrating that MegaVul is superior to BigVul for the SVA task.

Table 1 Comparison between Big-Vul and MegaVul

Statistic	Big-Vul	MegaVul
Number of CVE IDs	3,539	8,254
Number of Vul	10,547	17,380
Date range of crawled CVEs	2013/01 ∼ 2019/03	2006/01 ∼ 2023/10
Function Extract Method/(Quality)	Lizard/(Low)	Tree-sitter/(High)
Number of Crawled Websites	2	28
Number of Repositories	310	992
Code Integrality	Partial	Full
Number of CWE IDs	92	169
Number of Commits	4,058	9,019

We observed that the MegaVul dataset is not entirely based on the CVSS v3 standard, with a significant portion following the CVSS v2 standard. We scraped the CVSS v3 data from CVE and replaced the CVSS v2 scores in MegaVul with the CVSS v3 scores. Our final dataset contains 14,235 vulnerabilities. During the dataset splitting process, we ensured that vulnerabilities with the same CVE ID, despite having different codes, remained in the same dataset to maintain validity and fairness. We split the data into training, validation, and test sets in an 80%:10%:10% ratio.

4.3 Performance metrics

In our empirical study, We consider five performance metrics: Accuracy, Precision, Recall, F1 score, and Matthews correlation coefficient (MCC), which can provide a comprehensive performance evaluation.

For the SVA task, the first four metrics are common, while MCC is a widely used performance metric for datasets with the class imbalance issue. Since we need to predict four severity categories, we use macro-averaged metrics to present the final results. In the rest of the subsection, we show the details of calculating these performance metrics.

TP True Positive, indicating the number of samples in which the positive class is correctly classified. In the context of the SVA task, it represents a situation in which a model accurately identifies various levels of severity.

TN True Negative, indicating the number of samples in which the negative class is correctly classified. For each severity level, it represents the number of samples that were correctly classified as not of that severity level.

FN False Negative, indicating the number of samples in which the positive class is misclassified as the negative class. For each severity category, it represents the number of samples that were misclassified as not of that severity.

FP False Positive, indicating the number of samples in which the negative class is incorrectly classified as a positive class. For each severity level, it represents the number of samples that were misclassified to that severity level.

For each category i, these metrics can be represented as: TP_i, TN_i, FN_i, FP_i.

Accuracy Accuracy is the proportion of correctly predicted severity out of the total number.

$$\text{Accuracy} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i + FN_i + TN_i)} \tag{8}$$

Precision Precision rate refers to the proportion of all samples predicted as the positive class that are the positive class. The macro average precision is the average of the precision of each category, where N denotes the number of severity categories.

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} \tag{9}$$

$$\text{Precision}_{macro} = \frac{1}{N} \sum_i \text{Precision}_i \tag{10}$$

Recall The recall rate refers to the proportion of all samples that are positive classes, which are correctly predicted as positive classes. It measures the model's ability to identify all positive examples. The macro average recall rate is the average recall rate of each category and can be defined as follows.

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \tag{11}$$

$$\text{Recall}_{macro} = \frac{1}{N} \sum_i Recall_i \tag{12}$$

F1 score The F1 score is the geometric mean of Precision and Recall, representing a balance between these two metrics. The macro average F1 score is the average of the F1 scores across categories.

$$\text{F1-score} = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \tag{13}$$

$$\text{F1-score}_{macro} = \frac{1}{N} \sum_i F1_i \tag{14}$$

MCC MCC is a metric that takes into account TP, TN, FP, and FN. For multivariate classification, the macro mean of each class can be calculated from its MCC.

$$\text{MCC}_i = \frac{TP_i \cdot TN_i - FP_i \cdot FN_i}{\sqrt{(TP_i + FP_i)(TP_i + FN_i)(TN_i + FP_i)(TN_i + FN_i)}} \tag{15}$$

$$\mathrm{MCC}_{macro} = \frac{1}{N}\sum_{i} MCC_i \tag{16}$$

Notice that we use the macro versions of precision, recall, F1 score, and MCC in default for evaluating the performance of different methods.

4.4 Baseline methods

For our investigated SVA task, we selected five state-of-the-art baselines to demonstrate the competitiveness of PT-SVA. Among them, three methods proposed by Le et al. (2019) combine character and word features (i.e., vulnerability descriptions), including $\mathrm{CWM}_{\mathrm{NB}}$, $\mathrm{CWM}_{\mathrm{SVM}}$, and $\mathrm{CWM}_{\mathrm{XGB}}$. Moreover, two methods proposed by Le and Babar (2022) use vulnerable and non-vulnerable statements and extract context from vulnerable statements to develop function-level SVA models, including $\mathrm{Fun}_{\mathrm{RF}}$ and $\mathrm{Fun}_{\mathrm{LGBM}}$. We briefly introduce the characteristics of these baselines as follows.

$\mathrm{CWM}_{\mathrm{NB}}$ This baseline is proposed by Le et al. (2019) and considers Naive Bayes (NB) as the classifier. Specifically, NB is based on Bayesian decision theory, a simple and classic classification algorithm. This model only applies when the features are conditionally independent; otherwise, the classification performance is poor. In $\mathrm{CWM}_{\mathrm{NB}}$, NB does not optimize hyperparameters on the validation set.

$\mathrm{CWM}_{\mathrm{SVM}}$ This baseline is proposed by Le et al. (2019) and considers Support Vector Machine (SVM) as the classifier. SVM is a classification model whose key idea is mapping input space to a higher-dimensional space. The SVM model represents instances as points in space, separating instances of different classes with as wide a margin as possible. Then, it maps new instances into the same space and predicts their class based on which side of the margin they fall. In this baseline, the regularization values of SVM are optimized on the candidate value set $\{0.01, 0.1, 1, 10, 100\}$.

$\mathrm{CWM}_{\mathrm{XGB}}$ This baseline is proposed by Le et al. (2019) and considers Extreme Gradient Boosting (XGB) as the classifier. XGB is a boosting tree model that integrates many tree models to form a strong classifier. In this baseline, the three hyperparameters that need to be optimized for XGB are the number of trees, the maximum depth, and the maximum number of leaves.

$\mathrm{Fun}_{\mathrm{RF}}$ This baseline is proposed by Le and Babar (2022) and considers Random Forest (RF) as the classifier. RF consists of multiple decision trees, where each decision tree outputs a class label and the final classification is determined by majority voting. The core idea of the Random Forest is "ensemble learning," which integrates a fixed number of decision trees with different features to achieve high predictive performance and robustness. In this baseline, the hyperparameters are optimized as follows: the candidate values of the number of estimators are 100, 200, 300, 400, and 500; the candidate values of the maximum depth are 3, 5, 7, 9, and unlimited; and the candidate values of the number of leaf nodes are 100, 200, 300, and unlimited.

Fun_{LGBM} This baseline is proposed by Le and Babar (2022) and considers Light Gradient Boosting Machine (LGBM) as the classifier. LGBM is an algorithm based on Gradient Boosting Decision Trees (GBDT) that accelerates the training speed of GBDT models without compromising accuracy. It has advantages such as lower memory consumption and support for distributed computing, making it capable of handling massive datasets. In this baseline, the hyperparameter settings are the same as those in Fun_{RF}, except that the maximum number of leaves cannot be set to unlimited.

We did not select certain related SVA methods (Han et al. 2017; Sahin and Tosun 2019; Nakagawa et al. 2019; Gong et al. 2019; Sun et al. 2023; Hao et al. 2023) mentioned in the related work as baselines primarily because we could not find reproducible code for these methods, making implementation challenging.

To bridge the gap between pre-training tasks and downstream tasks, we introduce prompt tuning for the SVA task in our study. To investigate the contribution of prompt tuning to PT-SVA's performance, we further consider fine tuning-based methods as our baselines. Among these baselines, we consider current state-of-the-art pre-trained models, such as CodeT5 (Wang et al. 2021), T5 (Raffel et al. 2020), CodeBERT (Feng et al. 2020), GraphCodeBERT (Guo et al. 2020), and UniXcoder (Guo et al. 2022). For these baselines, we denote them as FT_{CT}, FT_{T5}, FT_{CB}, FT_{GCB}, and FT_{UC}, respectively. Specifically, these baselines focus on fine-tuning without using any prompts. Instead, we use the identifier <desc> as a prefix to distinguish bi-modal information. The input format is defined as:

$$X = X_{\text{code}} + < \text{desc} > + X_{\text{desc}}$$

where X denotes the input data, X_{code} denotes the source code, and X_{desc} denotes the vulnerability description.

4.5 Experimental settings

In our study, to implement prompt tuning, we integrated a hybrid template during the prompt tuning stage, which was rapidly constructed using the OpenPrompt framework (Ding et al. 2022). Following the approach of Wang et al. (2022), who were the first to introduce prompt tuning into software engineering tasks and achieved promising results, we adopt a similar strategy. Specifically, during training, we not only update the embeddings of the soft prompt tokens and verbalizer, but also update the model parameters.

Regarding the model hyperparameters, we used the default settings of CodeT5. We set the word embedding dimension and hidden size to 768, with 12 attention heads and 12 layers. All hyperparameters were optimized using AdamW (Diederik 2014), with an initial learning rate set to 5e-5. During training, the batch size was set to 32, and the maximum input sequence length was set to 512. To prevent overfitting, we employed an early stopping strategy (Prechelt 2002), halting training when the model's performance on the validation set declined for 10 consecutive epochs. The model with the best performance on the validation set was selected.

Due to the input length limitation for CodeT5, we conducted a length analysis of the source code and vulnerability descriptions in our constructed SVA dataset. We

calculated their average lengths and lengths at the 40-th, 60-th, 80-th, and 100-th percentiles, as shown in Table 2. To maximize the inclusion of vulnerability information, we truncated the bi-modal information based on word count, capping the source code at 384 words and the vulnerability description at 64 words.

All experiments were run on a computer equipped with an Intel(R) Core(TM) i5-13600K processor and a GeForce RTX 4090 GPU with 24 GB of memory.

5 Experimental results

5.1 RQ1. Effectiveness of PT-SVA

To demonstrate the effectiveness of our method, PT-SVA, we compare it with state-of-the-art baselines using widely adopted performance metrics, including Accuracy, Precision, Recall, F1 score, and MCC. All experiments for the baselines were conducted using their optimized hyperparameters.

The comparison results are shown in Table 3, with the best results highlighted in bold. From the table, it is clear that our proposed method outperforms all the baselines. Specifically, compared to the baselines, the F1 score performance of PT-SVA improves by 8.8% to 36.3%, MCC performance improves by 13.7% to 42.1%, Accuracy performance improves by 7.7% to 22.6%, Precision performance improves by 4.6% to 29.9%, and Recall performance improves by 13.1% to 37.7%.

We conjecture that the superior performance of our proposed method, compared to traditional SVA methods and the controlled methods based on the fine-tuning paradigm, can be attributed to the overall effectiveness of pre-trained language models and the use of prompt tuning. Traditional SVA methods typically do not consider bimodal information and focus solely on either source code or vulnerability descriptions. In contrast, our method integrates both types of vulnerability information, enabling a more comprehensive learning of vulnerability knowledge. Moreover, compared to the fine-tuning paradigm, the prompt tuning paradigm achieves better performance, likely due to its ability to facilitate more effective learning of vulnerability information and bridge the gap between the pre-training task and the SVA task.

Answer to RQ1: PT-SVA outperforms all baselines in terms of all the performance metrics. For example, the F1 score of PT-SVA improves by 8.8% to 36.3%, and MCC improves by 13.7% to 42.1%.

Table 2 Word count distribution of different types of vulnerability information

Statistics	Source code	Vulnerability description
Mean	245	42
40-th	96	30
60-th	179	39
80-th	366	53
100-th	3,944	403

Table 3 Performance comparison between PT-SVA and SVA baselines, with the best results for each metric highlighted in bold

Method	Accuracy	Precision	Recall	F1	MCC
CWM_{NB}	0.603	0.633	0.466	0.504	0.332
CWM_{SVM}	0.620	0.565	0.496	0.515	0.369
CWM_{XGB}	0.657	0.643	0.559	0.591	0.436
Fun_{RF}	0.542	0.449	0.313	0.316	0.152
Fun_{LGBM}	0.508	0.390	0.341	0.349	0.197
FT_{CT}	0.598	0.464	0.447	0.451	0.326
FT_{T5}	0.551	0.431	0.434	0.426	0.276
FT_{CB}	0.564	0.458	0.465	0.461	0.272
FT_{GCB}	0.577	0.473	0.435	0.448	0.286
FT_{UC}	0.525	0.413	0.438	0.411	0.263
PT-SVA	**0.734**	**0.689**	**0.690**	**0.679**	**0.573**

5.2 RQ2. Ablation study on using the bi-modal information

To enhance the training input for the SVA task, our goal is to capture vulnerability information from both source code and vulnerability description. To demonstrate the effectiveness of considering both types of vulnerability information (i.e., our customized bi-modal input setting), we aim to study the contribution of different modalities to PT-SVA in this RQ.

For PT-SVA, we considered two input modalities. We used "w/o desc" to indicate that no vulnerability description is used in the corresponding control method, and "w/o code" to indicate that no source code is used in the corresponding control method.

The performance of PT-SVA using different input modality settings is shown in Table 4, with the best results highlighted in bold. Specifically, When focusing solely on vulnerability descriptions (i.e., without source code), the performance of our method PT-SVA slightly decreases in all metrics in all metrics except Precision, which shows a slight improvement. However, when focusing solely on source code, the performance significantly drops. Accuracy decreases by 23.9%, Precision by 29.0%, Recall by 30.3%, F1 score by 30.0%, and MCC by 42.0%. These findings suggest that the vulnerability description contributes more to improving the performance of PT-SVA when compared to the source code. Therefore, the fusion of bi-modal information allows PT-SVA to achieve the best performance.

Figure 3 uses two cases to illustrate the prediction results of our method, PT-SVA, under different input modalities. In Case 1, when only the vulnerability description is considered, PT-SVA returned an incorrect prediction (the ground truth is "HIGH," but

Table 4 Comparison results between our proposed method PT-SVA and PT-SVA with different modality information, with the best results highlighted in bold

Setting	Accuracy	Precision	Recall	F1	MCC
w/o desc	0.495	0.399	0.387	0.379	0.153
w/o code	0.724	**0.744**	0.623	0.664	0.528
PT-SVA	**0.734**	0.689	**0.690**	**0.679**	**0.573**

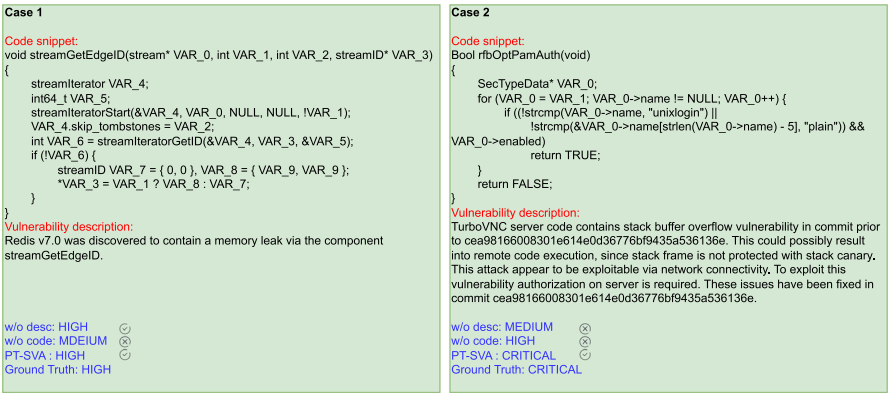

Fig. 3 Severity prediction results of PT-SVA with different input modalities on two vulnerabilities

it was predicted as "MEDIUM"). We conjecture that this discrepancy arises because the information contained in the vulnerability description alone is insufficient to extract adequate vulnerability details. However, when only the source code is considered, the vulnerability is correctly predicted. When both modalities are considered, the model can also correctly predict the severity. In Case 2, neither the source code alone nor the vulnerability description alone allows the model to correctly predict the vulnerability severity. However, when both modalities are considered simultaneously, the correct prediction can be achieved. This indicates that combining the source code and vulnerability description helps the model better learn vulnerability knowledge, compensating for the limitations of each modality individually and thereby improving the performance of SVA.

> **Answer to RQ2:** Compared to the source code, the vulnerability description has a higher impact on improving the performance of PT-SVA. The fusion of both modalities achieves the best performance, indicating that there is complementary information in these different modalities.

5.3 RQ3. Impact of different prompt settings

Designing prompt templates and verbalizers is still a challenging and open problem for prompt tuning. In this RQ, we aim to investigate the impact of our designed hybrid prompt templates and verbalizers on PT-SVA. To show the effectiveness of our settings, we design various types of prompt templates and verbalizers.

Impact of Templates To explore the impact of templates, we use (6) as our verbalizer. The types of prompt templates include hard prompts, soft prompts, and hybrid prompts. To investigate the prompt template impact, we designed two control methods that only consider hard prompts or soft prompts.

PT-SVA with hard prompt: This control method uses only the hard prompt template as the prompt template, manually adding natural language prompts to construct the template. Specifically, we use (2) as the hard prompt template and denote this method as w/hard.

PT-SVA with soft prompt: This control method uses only the soft prompt template as the prompt template, creating the template with soft tokens but initializing them with natural words from the hard prompts. Specifically, we use (3) as the soft prompt template and denote this method as w/soft.

In Table 5, we present the performance of PT-SVA under different prompt templates. Specifically, when only considering hard prompts (w/hard), the performance decreases in all metrics except Precision, which shows a slight improvement, with Accuracy, Recall, F1 score, and MCC decreasing by 7.0%, 10.1%, 7.3%, and 11.3%, respectively. We conjecture the reason is that hybrid prompts might extend the capability of the prompt templates, enabling better SVA performance. When only considering soft prompts (w/soft), the performance slightly decreases in all metrics except Precision, which shows a slight improvement. Accuracy, Recall, F1 score, and MCC decrease by 1.0%, 5.8%, 1.9%, and 4.4%, respectively. We conjecture this is because, in hybrid prompt templates, the inclusion of important task-related tokens in the prompts allows our method to better separate source code and vulnerability descriptions from the bi-modal information, thus resulting in better performance with hybrid prompts.

Impact of Verbalizers To investigate the impact of verbalizers, we use hybrid prompts as the prompt template, i.e., (4). We set up different one-to-one and one-to-many verbalizers, as shown in (5) and (6). We aim to investigate the impact of the number of mapping words in the verbalizer on the performance of our method, PL-SVA. In a one-to-many verbalizer, we first consider two mapping words and then observe the performance change when adding a mapping word. In Table 6, we present the impact of different verbalizers on the performance of our proposed method PT-SVA. When using two mapping words, Accuracy, Recall, F1 score and MCC improved by 3.6%, 9.6%, 5.2%, and 8.4%, respectively, compared to the method using one mapping word. However, when using three mapping words, performance declined: Accuracy, Recall, F1 score, and MCC decreased by 2.0%, 4.9%, 1.8%, and 4.9%, respectively. Based on the above results, We find that adding synonyms to the target labels to form a one-to-many verbalizer can moderately improve the model's performance. However, adding more mapping words does not always result in better performance. Adding too many words, especially unrelated ones, can interfere with the model and degrade performance.

Table 5 Comparison results between PT-SVA and the methods with different types of prompt templates, with the best results highlighted in bold

Setting	Accuracy	Precision	Recall	F1	MCC
w/hard	0.664	**0.707**	0.589	0.606	0.460
w/soft	0.724	0.702	0.632	0.660	0.529
PT-SVA	**0.734**	0.689	**0.690**	**0.679**	**0.573**

Table 6 Performance of different verbalizers on PT-SVA's severity classification, with the best results highlighted in bold

Verbalizer	Accuracy	Precision	Recall	F1	MCC
"LOW": ["low"] "MEDIUM": ["medium"] "HIGH": ["high"] "CRITICAL": ["critical"]	0.698	**0.713**	0.594	0.627	0.489
"LOW": ["low", "slight"] "MEDIUM": ["medium", "moderate"] "HIGH": ["high", "severe"] "CRITICAL": ["critical", "significant"]	**0.734**	0.689	**0.690**	**0.679**	**0.573**
"LOW": ["low", "slight", "small"] "MEDIUM": ["medium", "moderate", "middle"] "HIGH": ["high", "severe", "big"] "CRITICAL": ["critical", "significant", "urgent"]	0.714	0.708	0.641	0.661	0.524

Answer to RQ3: Our results indicate that using hybrid prompts achieves the best performance for PT-SVA compared to using only hard prompts or only soft prompts. Additionally, while a one-to-many verbalizer with extra mapping words can improve the performance of PT-SVA, adding more mapping words does not necessarily lead to better performance.

5.4 Discussions

5.4.1 Influence of training set size

Since the prompt-tuning paradigm is less dependent on the size of the training set compared to the pre-tuning paradigm (Lu et al. 2023; Ren et al. 2024; Wang et al. 2023, 2022), we aim to explore the impact of training set size on the performance of PT-SVA. In this experiment, we only change the size of the training set, while keeping the validation set and the test set unchanged. Specifically, we first extract 20% from the training set and then expand it to 40%, 60%, 80%, and 100% in sequence.

In this experiment, we primarily observe the value changes in the performance metrics of the F1 score and MCC. As shown in Fig. 4, the performance improves with the increase of the training set size. Specifically, when the training set size increases from 40% to 60%, the performance improves the fastest, with the F1 score increasing by 5.2% and the MCC increasing by 6.1%. The performance changes in other ranges were relatively smooth. Notably, the performance using only 20% of the training set even surpassed that of FT_{CT}. Therefore, we can conclude that using the prompt-tuning paradigm, PT-SVA performs well even on the small-scale training set, and the performance continues to improve as the size of the training set increases.

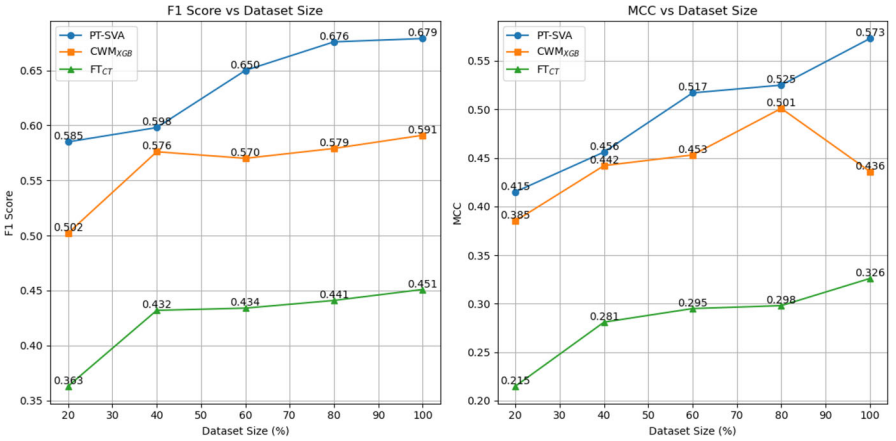

Fig. 4 Changes in F1 score and MCC of methods when training with different proportions of the training set

Finally, we select a representative method from the fine-tuning-based baselines (i.e., FT_{CT}) and SVA baselines (i.e., CWM_{XGB}) respectively, each with the best performance. We aim to analyze whether our proposed method could consistently outperform these two representative methods across different dataset sizes. The results indicate that, regardless of the dataset size, our method, PT-SVA, consistently demonstrates better performance than these two baselines.

5.4.2 Influence of pre-trained language models

The PLM used in our method is CodeT5. To evaluate the performance of our method with other PLMs, we kept the method details unchanged, replacing only the PLM and using the same metrics for comparison. We considered four other PLMs commonly used in downstream tasks: T5 (Raffel et al. 2020), CodeBERT (Feng et al. 2020), GraphCodeBERT (Guo et al. 2020) and UniXcoder (Guo et al. 2022). This discussion highlights the performance gap between our method and those using other PLMs.

As shown in Table 7, our method using CodeT5 outperforms those using other PLMs. Compared to the method using T5, Accuracy, Recall, F1 score, and MCC can

Table 7 Impact of Different Pre-trained Language Models on PT-SVA Performance, with the best-performing results shown in bold

PLM	Accuracy	Precision	Recall	F1	MCC
T5	0.662	0.697	0.574	0.617	0.423
CodeBERT	0.674	0.671	0.570	0.602	0.446
UniXcoder	0.716	0.682	0.658	0.663	0.526
GraphCodeBERT	0.693	**0.715**	0.640	0.656	0.494
CodeT5	**0.734**	0.689	**0.690**	**0.679**	**0.573**

be improved by 7.2%, 11.6%, 6.2%, and 15.0%, respectively. While T5 is pre-trained primarily on natural language texts, making it effective for tasks like machine translation and text generation, it is less suited for bi-modal tasks that involve both natural language and source code. When compared to the method using CodeBERT, Accuracy, Precision, Recall, F1 score, and MCC can be improved by 6.0%, 1.8%, 12.0%, 7.7%, and 12.7%, respectively. CodeBERT, although pre-trained on both code and natural language from GitHub, is based on the BERT architecture, which is not optimized for deep interactions between code and text. Compared to the method using UniXcoder, Accuracy, Precision, Recall, F1 score, and MCC can be improved by 1.8%, 0.7%, 3.2%, 1.6%, and 4.7%, respectively. While UniXcoder excels in code generation, code translation, and multilingual tasks, it has certain limitations in handling the complexities of SVA. Finally, compared to the method using GraphCodeBERT, Accuracy, Recall, F1 score, and MCC can be improved by 4.1%, 5%, 2.3%, and 7.9%, respectively. GraphCodeBERT, which focuses on capturing syntactic and semantic relationships in code using graph neural networks, performs well in structured code representation but has limitations in text processing.

5.4.3 Influence of the number of soft prompt tokens

As discussed in Section 3.2, in software prompts/hybrid prompts, we simply replace the handcrafted text in hard prompts with an equal number of soft tokens by following previous work (Wang et al. 2023). In this subsection, we want to further investigate the performance impact of the number of soft prompt tokens on PT-SVA.

Since we found that hybrid prompts achieve the best performance in PT-SVA (discussed in Section 5.3), we focus on analyzing the impact of the soft prompt token number in hybrid prompts. In this experiment, in addition to using "Classify the severity:" to initialize the soft tokens, we also considered other similar descriptions, such as "severity:", "the severity:", and "Classify the vulnerability severity:". These descriptions correspond to different soft prompt token numbers and we use the corresponding description for soft token initialization. Then we conducted experiments based on different soft prompt token numbers, and the detailed results are shown in Table 8. From the table, we observe that the number of soft prompt tokens does indeed have a certain impact on the performance of PT-SVA. However, in our experiments, we find that using "Classify the severity:" to initialize the soft tokens in our hybrid prompt yields better performance across most metrics, except for the Precision metric.

Table 8 The performance impact of different soft token numbers on PT-SVA with the best-performing results highlighted in bold

[SOFT] with initial description	Accuracy	Precision	Recall	F1	MCC
"severity:"	0.693	0.740	0.647	0.673	0.494
"the severity:"	0.696	0.736	0.656	0.672	0.502
"Classify the severity:"	**0.734**	0.689	**0.690**	**0.679**	**0.573**
"Classify the vulnerability severity:"	0.718	**0.775**	0.619	0.653	0.537

6 Threats to validity

In this subsection, we analyze potential threats to the validity of our research results.

Internal Threats This threat pertains to the potential issues in implementing PT-SVA and the baselines. To alleviate this threat, we carefully checked these implementations through code review and testing. Additionally, there are many model hyperparameters (such as the optimizer and learning rat) and other better prompt designs. Finding the optimal hyperparameter settings and the prompt design is a challenging task. However, our experimental results based on the current hyperparameter setting and prompt design can still outperform the baselines and traditional fine-tuning paradigms. Moreover, when replicating the baselines, we performed hyperparameter optimization of these methods to ensure a fair comparison.

External Threats The first threat arises from the selected dataset. To alleviate this threat, we selected the high-quality dataset MegaVul, which covers a broad time range (i.e., from 2006 to 2023) and includes a large number of vulnerabilities (over 10,000). We also crawled CVSS scores from the CVSS v3 standard, ultimately constructing a new SVA dataset. The second threat is currently limited to vulnerabilities in C/C++ code. However, we believe that our method is scalable to other programming languages, such as Java and Python. The methodology can be extended by first collecting relevant datasets for the target language and then applying our method to constructing corresponding vulnerability assessment models.

Construct Threats This threat is related to our considered performance metrics. To alleviate this threat, we considered a wide range of performance metrics, which can provide a comprehensive evaluation on PT-SVA and baselines.

7 Related work

7.1 Software vulnerability assessment

Software vulnerability assessment is the process of identifying, analyzing, and prioritizing vulnerabilities in software systems. This assessment is crucial for maintaining the security and reliability of software applications, as it helps in identifying potential weaknesses that could be exploited by attackers. As the number of software vulnerabilities increases with the development of network and software technologies, relying solely on expert knowledge to evaluate software vulnerabilities one by one becomes overly slow. Therefore, automated methods or models are needed to conduct software vulnerability assessments. Previous studies on software vulnerability assessment are primarily based on either vulnerability descriptions or source code.

Vulnerability description-based SVA A vulnerability description provides a detailed account of a software flaw, including its nature, potential impact, and the conditions under which it can be exploited. Previous studies show that vulnerability descriptions can be used for the SVA task. Han et al. (2017) transformed the SVA task into a text

classification task by capturing word-level features through word embeddings and then designed a shallow CNN to capture sentence-level features for severity classification. Sahin and Tosun (2019) extended the study of Han et al. (2017) by using LSTM and XGBoost models. Nakagawa et al. (2019) proposed a different method through a character-level CNN architecture. They used the one-hot encoded vectors of each character in the CVE description as input vectors. Le et al. (2019) proposed a systematic method combining character and word features, using time-based k-fold cross-validation for model selection and using vulnerability descriptions for severity classification. Gong et al. (2019) proposed a multi-task machine learning method that eliminates the need for balanced data by jointly predicting multiple vulnerability features based on vulnerability descriptions. Sun et al. (2023) argued that there are some useless elements in vulnerability descriptions, so they proposed using vulnerability elements instead of complete descriptions to assess severity.

Source code-based SVA In addition to using vulnerability descriptions for SVA, recent research has also explored methods based on analyzing the source code of vulnerabilities. Le and Babar (2022) used data-driven models to automate the function-level SVA task, applying fine-grained vulnerable code statements in the assessment model. Their method used vulnerable and non-vulnerable statements to capture the context related to vulnerable statements and gather additional information about vulnerabilities from the context. Hao et al. (2023) utilized a graph attention neural network algorithm to extract key vulnerability features from function call graphs and vulnerability attribute graphs, leveraging the information from both graphs to classify vulnerability severity.

Recently, Pan et al. (2024) addressed the challenge of early vulnerability assessment by focusing on software vulnerability-related issue reports to provide timely prioritization of critical vulnerabilities. Motivated by the need to reduce delays in CVSS score updates and improve the practical value of automated vulnerability assessment, they proposed a prompt-based model proEVA, which leverages strong associations between CVSS metrics for accurate predictions. A key feature of their method is the use of a curriculum-learning schedule to progressively train the model, allowing it to better capture the relationships between CVSS metrics.

Different from previous SVA studies on vulnerability description or source code, we are the first to apply LLM to this task and achieve promising results. Specifically, we propose a prompt tuning-based method PT-SVA. To improve the performance, we design a hybrid prompt template by considering both types of vulnerability information (i.e., source code and vulnerability description), while previous SVA studies only consider one type of vulnerability information. Then our SVA model and the hybrid prompt template are fine-tuned using the pre-trained language model CodeT5 via prompt tuning. To evaluate the effectiveness of PT-SVA, we also construct a new SVA dataset with CVSS v3 standard.

Notice that Pan et al. (2024) also employed prompt tuning to address the SVA problem, but there are significant differences. Their work primarily focuses on leveraging associations between CVSS metrics to predict the CVSS vector, with an emphasis on early vulnerability assessment using vulnerability-related issue reports and prioritization. In contrast, our method focuses on fusing bi-modal information from source code and vulnerability descriptions, using prompt tuning to process both the code and

textual descriptions simultaneously for more accurate vulnerability severity prediction. Additionally, we introduce a hybrid prompt design specifically tailored to fully exploit the distinct characteristics of these two input modalities.

7.2 Prompt tuning for software engineering

Applying fine-tuning to downstream tasks is challenging due to the significant gap between pre-training tasks and downstream tasks. The emergence of prompt tuning helps bridge this gap, making prompt tuning a popular paradigm for recent LLM4SE (LLM for software engineering) studies. For example, Yu et al. (2023) used prompt tuning to detect reentrancy and timestamp dependency vulnerabilities in smart contracts. Ren et al. (2024) used prompt tuning to guide PLM in learning vulnerability information, introducing a reinforcement learning reward mechanism that guides the behavior of vulnerability detection through rewards and penalties. Yang et al. (2024a) leveraged prompt tuning and utilized dual-modal information (i.e., code snippets and problem descriptions) to automatically generate question titles. They collected question posts in six popular programming languages from Stack Overflow. Compared to baselines, their method demonstrated competitive performance, highlighting the advantages of prompt tuning in question generation. Wang et al. (2022) applied prompt tuning to three code intelligence tasks (i.e., defect prediction, code summarization, and code translation), which demonstrates the potential of prompt tuning for SE tasks.

To the best of our knowledge, We are the first to apply prompt tuning to bi-modal inputs (both source code and vulnerability descriptions) for software vulnerability assessment task and propose a corresponding method. Empirical studies on our constructed SVA dataset also show the effectiveness of PT-SVA and the customization components (such as the hybrid prompt template, both types of vulnerability information, and prompt tuning) of PT-SVA.

8 Conclusion and future work

In our study, we propose a novel SVA method, PT-SVA, which predicts vulnerability severity based on source code and vulnerability descriptions. PT-SVA employs hybrid prompts to bridge the gap between PLMs and software vulnerability assessment tasks. We also constructed a high-quality SVA dataset adhering to the CVSS v3 standard, containing over 14,000 vulnerabilities, to facilitate future SVA research. Comparative analysis with state-of-the-art SVA methods showed that PT-SVA outperformed both baseline and fine-tuning-based methods, demonstrating its effectiveness. Additionally, our ablation experiment confirmed the validity of the component customization in PT-SVA.

In future work, we want to include more vulnerabilities from various programming languages to expand our SVA dataset and explore the generalization capabilities of PT-SVA. Furthermore, we want to investigate more effective prompt settings for the SVA task, such as prompt templates and verbalizers.

Acknowledgements Jiyu Wang and Xiang Chen have contributed equally to this work and they are co-first authors. Xiang Chen is the corresponding author. This research was partially supported by the National Natural Science Foundation of China (Grant no. 61202006), the Open Project of State Key Laboratory for Novel Software Technology at Nanjing University under (Grant No. KFKT2024B21) and the Postgraduate Research & Practice Innovation Program of Jiangsu Province (Grant no. SJCX25_2002 and SJCX24_2018).

Author Contributions **Jiyu Wang:** Data curation, Software, Validation, Conceptualization, Methodology, Writing -review & editing. **Xiang Chen:** Conceptualization, Methodology, Writing -review & editing, Supervision. **Wenlong Pei:** Conceptualization, Data curation, Software. **Shaoyu Yang:** Conceptualization, Data curation, Software.

Declarations

Competing interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Apache Log4j2. https://logging.apache.org/log4j/2.12.x/

Cai, Z., Cai, Y., Chen, X., Lu, G., Pei, W., Zhao, J.: Csvd-tf: cross-project software vulnerability detection with tradaboost by fusing expert metrics and semantic metrics. J. Syst. Softw. **213**, 112038 (2024)

Chen, X., Zhao, Y., Cui, Z., Meng, G., Liu, Y., Wang, Z.: Large-scale empirical studies on effort-aware security vulnerability prediction methods. IEEE Trans. Reliab. **69**(1), 70–87 (2019)

Cheng, J., Dong, L., Lapata, M.: Long short-term memory-networks for machine reading. In: 2016 Conference on Empirical Methods in Natural Language Processing, pp. 551–561 (2016). Association for Computational Linguistics

Ciniselli, M., Martin-Lopez, A., Bavota, G.: On the generalizability of deep learning-based code completion across programming language versions. In: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, pp. 99–111 (2024)

CVSS : "Common Vulnerability Scoring System,". https://www.first.org/cvss

Diederik, P.K.: Adam: A method for stochastic optimization. (2014)

Ding, N., Hu, S., Zhao, W., Chen, Y., Liu, Z., Zheng, H., Sun, M.: Openprompt: An open-source framework for prompt-learning. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, pp. 105–113 (2022)

Fan, J., Li, Y., Wang, S., Nguyen, T.N.: Ac/c++ code vulnerability dataset with code changes and cve summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512 (2020)

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., *et al.*: Codebert: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547 (2020)

Feutrill, A., Ranathunga, D., Yarom, Y., Roughan, M.: The effect of common vulnerability scoring system metrics on vulnerability exploit delay. In: 2018 Sixth International Symposium on Computing and Networking (CANDAR), pp. 1–10 (2018). IEEE

Garg, A., Degiovanni, R., Jimenez, M., Cordy, M., Papadakis, M., Le Traon, Y.: Learning from what we know: how to perform vulnerability prediction using noisy historical data. Empir. Softw. Eng. **27**(7), 169 (2022)

Gong, X., Xing, Z., Li, X., Feng, Z., Han, Z.: Joint prediction of multiple vulnerability characteristics through multi-task learning. In: 2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS), pp. 31–40 (2019). IEEE

Gu, Y., Han, X., Liu, Z., Huang, M.: Ppt: Pre-trained prompt tuning for few-shot learning. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8410–8423 (2022)

Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J.: Unixcoder: Unified cross-modal pre-training for code representation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 7212–7225 (2022)

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Shujie, L., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al.: Graphcodebert: Pre-training code representations with data flow. In: International Conference on Learning Representations, (2020)

Guo, Y., Shi, H., Kumar, A., Grauman, K., Rosing, T., Feris, R.: Spottune: transfer learning through adaptive fine-tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4805–4814 (2019)

Han, Z., Li, X., Xing, Z., Liu, H., Feng, Z.: Learning to predict severity of software vulnerability using only vulnerability description. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 125–136 (2017). IEEE

Han, X., Zhang, Z., Ding, N., Gu, Y., Liu, X., Huo, Y., Qiu, J., Yao, Y., Zhang, A., Zhang, L., et al.: Pre-trained models: past, present and future. AI Open. **2**, 225–250 (2021)

Han, X., Zhao, W., Ding, N., Liu, Z., Sun, M.: Ptr: Prompt tuning with rules for text classification. AI Open. **3**, 182–192 (2022)

Hao, J., Luo, S., Pan, L.: A novel vulnerability severity assessment method for source code based on a graph neural network. Inf. Softw. Technol. **161**, 107247 (2023)

Howard, J., Ruder, S.: Universal language model fine-tuning for text classification. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 328–339 (2018)

Kudjo, P.K., Chen, J., Mensah, S., Amankwah, R., Kudjo, C.: The effect of bellwether analysis on software vulnerability severity prediction models. Softw. Qual. J. **28**, 1413–1446 (2020)

Le, T.H.M., Babar, M.A.: On the use of fine-grained vulnerable code statements for software vulnerability assessment models. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 621–633 (2022)

Le, T.H.M., Hin, D., Croft, R., Babar, M.A.: Deepcva: Automated commit-level vulnerability assessment with deep multi-task learning. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 717–729 (2021). IEEE

Le, T.H.M., Sabir, B., Babar, M.A.: Automated software vulnerability assessment with concept drift. In: 2019 IEEE/ACM 16th international conference on mining software repositories (MSR), pp. 371–382 (2019). IEEE

Lester, B., Al-Rfou, R., Constant, N.: The power of scale for parameter-efficient prompt tuning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 3045–3059 (2021)

Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pp. 4582–4597 (2021)

Li, X., Ren, X., Xue, Y., Xing, Z., Sun, J.: Prediction of vulnerability characteristics based on vulnerability description and prompt learning. In: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 604–615 (2023). IEEE

Liu, P., Qiu, X., Huang, X.: Recurrent neural network for text classification with multi-task learning. In: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, pp. 2873–2879 (2016)

Liu, K., Yang, G., Chen, X., Yu, C.: Sotitle: A transformer-based post title generation approach for stack overflow. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 577–588 (2022). IEEE

Liu, K., Zhou, Y., Wang, Q., Zhu, X.: Vulnerability severity prediction with deep neural network. In: 2019 5th International Conference on Big Data and Information Analytics (BigDIA), pp. 114–119 (2019). IEEE

Liu, K., Chen, X., Chen, C., Xie, X., Cui, Z.: Automated question title reformulation by mining modification logs from stack overflow. IEEE Trans. Softw. Eng. **49**(9), 4390–4410 (2023)

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput. Surv. **55**(9), 1–35 (2023)

Liu, Y., Wang, C., Ma, Y.: Dl4sc: a novel deep learning-based vulnerability detection framework for smart contracts. Autom. Softw. Eng. **31**(1), 24 (2024)

Liu, C., Chen, X., Li, X., Xue, Y.: Making vulnerability prediction more practical: prediction, categorization, and localization. Inf. Softw. Technol. **171**, 107458 (2024)

Lu, G., Ju, X., Chen, X., Yang, S., Chen, L., Shen, H.: Assessing the effectiveness of vulnerability detection via prompt tuning: An empirical study. In: 2023 30th Asia-Pacific Software Engineering Conference (APSEC), pp. 415–424 (2023). IEEE

Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z.: Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. J. Syst. Software. **212**, 112031 (2024)

Nakagawa, S., Nagai, T., Kanehara, H., Furumoto, K., Takita, M., Shiraishi, Y., Takahashi, T., Mohri, M., Takano, Y., Morii, M.: Character-level convolutional neural network for predicting severity of software vulnerability from vulnerability description. IEICE Trans. Inf. Syst. **102**(9), 1679–1682 (2019)

Ni, C., Shen, L., Yang, X., Zhu, Y., Wang, S.: Megavul: Ac/c++ vulnerability dataset with comprehensive code representations. In: 2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR), pp. 738–742 (2024). IEEE

Nie, E., Liang, S., Schmid, H., Schütze, H.: Cross-lingual retrieval augmented prompt for low-resource languages. In: The 61st Annual Meeting Of The Association For Computational Linguistics (2023)

Niu, C., Li, C., Ng, V., Chen, D., Ge, J., Luo, B.: An empirical comparison of pre-trained models of source code. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2136–2148 (2023). IEEE

Pan, S., Bao, L., Zhou, J., Hu, X., Xia, X., Li, S.: Towards more practical automation of vulnerability assessment. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)

Prechelt, L.: Early stopping-but when? In: Neural Networks: Tricks of the Trade, pp. 55–69 (2002). Springer, Berlin, Heidelberg

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. **21**(140), 1–67 (2020)

Ren, Z., Ju, X., Chen, X., Shen, H.: Prorlearn: boosting prompt tuning-based vulnerability detection by reinforcement learning. Autom. Softw. Eng. **31**(2), 38 (2024)

Ruan, X., Yu, Y., Ma, W., Cai, B.: Prompt learning for developing software exploits. In: Proceedings of the 14th Asia-Pacific Symposium on Internetware, pp. 154–164 (2023)

Sahin, S.E., Tosun, A.: A conceptual replication on predicting the severity of software vulnerabilities. In: Proceedings of the 23rd International Conference on Evaluation and Assessment in Software Engineering, pp. 244–250 (2019)

Schick, T., Schütze, H.: Exploiting cloze-questions for few-shot text classification and natural language inference. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, pp. 255–269 (2021)

Shen, Z., Liu, Z., Qin, J., Savvides, M., Cheng, K.-T.: Partial is better than all: Revisiting fine-tuning strategy for few-shot learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 9594–9602 (2021)

Sun, X., Ye, Z., Bo, L., Wu, X., Wei, Y., Zhang, T., Li, B.: Automatic software vulnerability assessment by extracting vulnerability elements. J. Syst. Softw. **204**, 111790 (2023)

Tsimpoukelli, M., Menick, J.L., Cabi, S., Eslami, S., Vinyals, O., Hill, F.: Multimodal few-shot learning with frozen language models. Adv. Neural Inf. Process. Syst. **34**, 200–212 (2021)

Wang, Y., Wang, W., Joty, S., Hoi, S.C.: Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 8696–8708 (2021)

Wang, C., Yang, Y., Gao, C., Peng, Y., Zhang, H., Lyu, M.R.: No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 382–394 (2022)

Wang, C., Yang, Y., Gao, C., Peng, Y., Zhang, H., Lyu, M.R.: Prompt tuning in code intelligence: an experimental evaluation. IEEE Trans. Softw. Eng. **49**(11), 4869–4885 (2023)

Wang, R., Xu, S., Ji, X., Tian, Y., Gong, L., Wang, K.: An extensive study of the effects of different deep learning models on code vulnerability detection in python code. Autom. Softw. Eng. **31**(1), 15 (2024)

Yang, G., Chen, X., Zhou, Y., Yu, C.: Dualsc: Automatic generation and summarization of shellcode via transformer and dual learning. In: 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 361–372 (2022). IEEE

Yang, G., Zhou, Y., Chen, X., Zhang, X., Han, T., Chen, T.: Exploitgen: template-augmented exploit code generation based on codebert. J. Syst. Softw. **197**, 111577 (2023)
Yang, S., Chen, X., Liu, K., Yang, G., Yu, C.: Automatic bi-modal question title generation for stack overflow with prompt learning. Empir. Softw. Eng. **29**(3), 63 (2024)
Yang, G., Zhou, Y., Yang, W., Yue, T., Chen, X., Chen, T.: How important are good method names in neural code generation? a model robustness perspective. ACM Trans. Softw. Eng. Methodol. **33**(3), 1–35 (2024)
Yu, L., Lu, J., Liu, X., Yang, L., Zhang, F., Ma, J.: Pscvfinder: A prompt-tuning based framework for smart contract vulnerability detection. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pp. 556–567 (2023). IEEE
Zhang, N., Li, L., Chen, X., Deng, S., Bi, Z., Tan, C., Huang, F., Chen, H.: Differentiable prompt makes pre-trained language models better few-shot learners. In: International Conference on Learning Representations (2021)
Zheng, W., Gao, J., Wu, X., Liu, F., Xun, Y., Liu, G., Chen, X.: The impact factors on the performance of machine learning-based vulnerability detection: a comparative study. J. Syst. Softw. **168**, 110659 (2020)
Zhong, Z., Friedman, D., Chen, D.: Factual probing is [mask]: Learning vs. learning to recall. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 5017–5033 (2021)

Authors and Affiliations

Jiyu Wang[1] · Xiang Chen[1,2] · Wenlong Pei[1] · Shaoyu Yang[1]

✉ Xiang Chen
xchencs@ntu.edu.cn

Jiyu Wang
jyu.wang@outlook.com

Wenlong Pei
wl.pei@outlook.com

Shaoyu Yang
shaoyuyoung@gmail.com

[1] School of Artificial Intelligence and Computer Science, Nantong University, Nantong, Jiangsu, China

[2] State Key Lab. for Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China